PixelFix Games — Requisitos e infraestrutura

Data: 24/10/2026. Documento elaborado por análise do código-fonte do projeto.

1. Objetivo e escopo

Sistema web de gestão de assistência técnica de consoles. Organiza clientes, equipamentos e agendamentos. A interface utiliza HTML, CSS e JavaScript; a API utiliza Node.js, Express e Sequelize com MySQL. Não há implementação de ordens de serviço, pagamentos, orçamento ou controle de estoque.

A indicação **implementado** descreve comportamento identificado no código, sem representar homologação em execução. Itens **propostos** são melhorias e não funcionalidades disponíveis. Não existem perfis de administrador ou funcionário: as rotas privadas exigem token, mas não restringem os registros ao cliente autenticado.

2. Requisitos funcionais

ID	Requisito	Evidência / critério de aceitação
RF01	Permitir cadastro público de clientes.	POST /cliente recebe nome, email, senha, cpf e telefone; retorna 201 no sucesso e 400 quando faltam campos.
RF02	Autenticar por e-mail e senha.	POST /login retorna token e nome quando as credenciais conferem; rejeita credenciais ausentes ou incorretas.
RF03	Controlar acesso às operações privadas.	Middleware exige token válido no cabeçalho Authorization; ausência, token inválido ou expirado resulta em 401.
RF04	Listar clientes.	GET /clientes retorna código, nome, e-mail, CPF descriptografado e telefone, sem senha.
RF05	Consultar cliente pelo código.	GET /cliente/:id retorna os dados ou 404 quando não encontrado.
RF06	Atualizar parcialmente os dados do cliente.	PUT /cliente/:id aceita campos preenchidos; mantém os demais e rejeita atualização vazia.
RF07	Excluir cliente.	DELETE /cliente/:id remove o registro ou retorna 404; a interface solicita confirmação.
RF08	Cadastrar console vinculado a cliente.	POST /console recebe modelo, marca, numeroSerie, tipo e idCliente; rejeita campos ausentes.
RF09	Listar consoles.	GET /consoles retorna dados do console, número de série descriptografado e código do cliente.
RF10	Cadastrar agendamento.	POST /agendamento recebe data, hora, idCliente e idConsole; retorna 201 quando gravado.
RF11	Listar agendamentos.	GET /agendamentos retorna código, data, hora e vínculos com cliente e console.
RF12	Atualizar parcialmente agendamento.	PUT /agendamento/:id altera campos informados; rejeita ID inválido e solicitação vazia; retorna 404 se não existe.

ID	Requisito	Evidência / critério de aceitação
RF13	Apresentar os resultados das operações na interface.	Formulários exibem mensagens da API e as telas de listagem apresentam tabelas.
RF14	Armazenar a sessão no navegador.	Login grava token no localStorage; as telas privadas enviam esse token e tratam resposta 401.

RF01–RF14: implementados. Fontes: backend/index.js, backend/controller, backend/middleware e frontend/js. Não há rotas para atualizar/excluir consoles ou excluir agendamentos.

3. Requisitos não funcionais

ID	Requisito	Situação e critério
RNF01	Comunicação HTTP com JSON.	Implementado: API recebe JSON, responde em JSON e usa códigos HTTP conforme o resultado.
RNF02	Persistência relacional.	Implementado: Sequelize e MySQL mantêm clientes, consoles e agendamentos com chaves estrangeiras.
RNF03	Sessão com prazo definido.	Implementado: token expira duas horas após o login e é validado na API.
RNF04	Criptografia dos campos definidos no projeto.	Implementado: senha, CPF e número de série são gravados com AES/CryptoJS. A chave está fixa no código; isso não equivale a uma solução de segurança pronta para produção.
RNF05	Organização modular.	Implementado: separação entre interface, controllers, models, conexão e middleware.
RNF06	Informar falhas ao usuário.	Implementado: respostas de validação, autenticação, registro inexistente e erro interno; falhas de rede são tratadas na interface.
RNF07	Identidade visual e adaptação à tela.	CSS com rosa, roxo e dourado, mantendo o layout simples original. A adaptação para dispositivos menores ainda precisa ser validada.
RNF08	Acesso por teclado.	Proposto: garantir foco visível e validar a navegação por teclado. Não houve auditoria de acessibilidade.
RNF09	Fortalecer autenticação e proteção de dados.	Proposto: hash de senha apropriado, segredos fora do código, HTTPS, autorização por registro e remoção de logs de dados pessoais.
RNF10	Validar conteúdo de forma consistente.	Proposto: validar CPF numérico e dígitos verificadores, e-mail, datas e dados no backend; escapar conteúdo em todas as tabelas.
RNF11	Definir desempenho e disponibilidade mensuráveis.	Proposto: estabelecer volume de usuários, meta de latência e disponibilidade e validar com testes de carga. O projeto não contém medições que sustentem metas numéricas.
RNF12	Recuperar dados após falha.	Proposto: agendar backups e testar restauração. O dump presente é uma cópia pontual, não uma rotina automatizada.

4. Regras de negócio

ID	Regra	Aplicação atual
RN01	Todo cliente precisa de nome, e-mail, senha, CPF e telefone no cadastro.	Validação de presença no controller e campos não nulos no model.

ID	Regra	Aplicação atual
RN 02	CPF deve ter comprimento de 11 caracteres.	Cadastro e atualização verificam comprimento; não conferem somente números nem dígitos verificadores.
RN 03	Credenciais válidas geram uma sessão de duas horas.	Login compara a senha informada com a senha descriptografada; middleware verifica expiração.
RN 04	Cada console pertence a um cliente existente.	idCliente obrigatório e chave estrangeira no banco; cliente pode ter vários consoles.
RN 05	Cada agendamento precisa de data, horário, cliente e console existentes.	Campos obrigatórios e chaves estrangeiras; cliente e console podem ter vários agendamentos.
RN 06	A atualização parcial preserva campos não preenchidos.	Controllers de cliente e agendamento ignoram campos vazios e rejeitam operação sem alterações.
RN 07	O código do agendamento e os vínculos alterados devem ser inteiros positivos.	Validação no controller de atualização de agendamento.
RN 08	A exclusão do cliente remove registros dependentes.	Relacionamentos e dump SQL configuram exclusão em cascata de consoles e agendamentos; exclusão de console no banco também remove seus agendamentos.
RN 09	Códigos dos registros são gerados automaticamente.	Chaves primárias inteiras com autoIncrement.
RN 10	A senha não deve aparecer nas consultas de cliente.	Listagem e consulta constroem respostas sem o campo senha.

Regras propostas, ainda não implementadas

- RN11: impedir agendamentos em horários ocupados, conforme capacidade e duração a definir.

- RN12: impedir datas passadas e horários fora do expediente, após definição do calendário de atendimento.

- RN13: verificar se o console agendado pertence ao cliente informado. As chaves estrangeiras verificam existência, mas não essa correspondência.

- RN14: definir e aplicar unicidade de e-mail, CPF e número de série, conforme política do negócio. Não existem restrições de unicidade para esses campos.

- RN15: definir perfis e permissões para consultar, alterar e excluir registros; hoje qualquer token aceito libera as rotas privadas.

- RN16: definir as categorias aceitas para tipo de console. Atualmente o campo é texto livre.

5. Requisitos de infraestrutura

ID	Componente	Configuração identificada / necessidade
RI 01	Ambiente de execução	Node.js e npm para instalar dependências e executar backend/index.js. A versão mínima não foi declarada em package.json; deve ser definida e homologada.
RI 02	Dependências	Express ^5.3.0, cors ^2.8.6, crypto-js ^4.2.0, mysql2 ^3.24.6 e sequelize ^6.37.8, conforme package.json.
RI 03	Banco de dados	MySQL local na porta 3306, schema a2. O dump foi exportado de MySQL 8.0.40; compatibilidade com outras versões requer validação.

ID	Componente	Configuração identificada / necessidade
RI 04	Credenciais	A conexão utiliza usuário e senha fixos em backend/db/conn.js. O banco deve existir e a conta precisa ter acesso às tabelas e à sincronização. Para produção, propor conta restrita e configuração por ambiente.
RI 05	Servidor HTTP da API	Endereço atual http://localhost:3000, vinculado a localhost. O código atual atende o ambiente local; acesso remoto exige alteração de endereço e configuração de rede.
RI 06	Interface web	Navegador com JavaScript, fetch e localStorage; frontend estático, sem etapa de build. Um servidor estático local é recomendado; a API não serve os arquivos HTML.
RI 07	Comunicação entre componentes	Os scripts usam URLs fixas para localhost:3000 e a API habilita cors(). A porta do servidor estático pode variar; manter a mesma origem entre telas para compartilhar localStorage.
RI 08	Estrutura de armazenamento	Preservar backend, frontend e diagramas, incluindo HTML, JavaScript, CSS e arquivos SQL. Nenhum serviço de armazenamento externo é exigido pelo código.
RI 09	Capacidade de hardware	CPU, memória, disco e número de usuários simultâneos não foram especificados nem medidos. Dimensionar conforme volume de registros e testes; não há mínimo comprovado.
RI 10	Backup e restauração	Há dump SQL em diagramas/Dump20261022.sql. Proposto: backups periódicos protegidos e restauração validada; importar o dump pode substituir tabelas existentes.
RI 11	Hospedagem em produção	Proposto: HTTPS, configuração externa de URLs/segredos, restrição de CORS, supervisor de processo, logs sem dados pessoais e banco com acesso controlado. Não configurado no projeto.

Execução local

1. Disponibilizar MySQL, criar o schema a2 e conferir a conexão em backend/db/conn.js.

2. Na pasta backend, executar `npm install` quando as dependências não estiverem instaladas.

3. Executar `node index.js`. O servidor usa conn.sync() antes de escutar na porta 3000; não é obrigatório importar dados de exemplo para criar as tabelas.

4. Servir a pasta frontend por um servidor estático e abrir index.html no navegador.

5. Cadastrar um cliente, efetuar login e utilizar as operações disponíveis.

O arquivo backend/sync.js usa `force:true`: sua execução apaga e recria tabelas. Não faz parte da inicialização normal e não deve ser executado para preservar dados existentes.

6. Rastreabilidade

Artefato	Responsabilidade
backend/index.js	Endereços, rotas públicas/privadas, middlewares e inicialização.
backend/controller/*.js	Validações, autenticação e operações de dados.
backend/middleware/auth.middleware.js	Validade e conteúdo do token.
backend/models/*.js	Campos, tabelas, vínculos e cascatas.
backend/db/conn.js e backend/package.json	Conexão e dependências da infraestrutura.
frontend/html, frontend/js e frontend/index.html	Telas, formulários, sessão e consumo da API.
frontend/css/style.css	Paleta visual, campos e tabelas no layout original.

Artefato	Responsabilidade
diagramas/Dump20261022.sql	Estrutura exportada do banco e dados de exemplo.

7. Limites da análise

A documentação foi produzida a partir dos arquivos disponíveis. Não foram executadas operações contra o MySQL nem validado o sistema com usuários reais. Os requisitos propostos precisam de definição do negócio antes de implementação.